SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration: 1 Hour
Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Mohammed Falah Miya / 1911260265 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Student Name	Mohammed Falah Miya
Registration No.	1911260265
Course	Artificial Intelligence (CSA17)

Scenario Based Assignment – Answers (BL3: Apply)

All five questions come under CO2, which is BL3 (Apply). Every answer therefore applies the algorithm to the given scenario with a small worked example, names the key issues, and ends with a justified decision.

Question 1 – Drone delivery in a flood region (Greedy vs A*)

Understanding and key issues: The environment is dynamic (roads get blocked), information is partial, costs vary with terrain and weather, and time and safety are critical. Model the map as a graph: nodes are waypoints, edges are flight legs. g(n) is the real cost so far (time plus a weather/risk penalty). h(n) is the straight-line distance to the delivery point.

i. Greedy Best-First Search (f(n) = h(n))

Greedy always expands the node that *looks* closest to the goal, so the drone flies straight toward the delivery point.

- *Strengths:* very fast, little memory, cheap to re-run after each update, so it reacts quickly.
- *Weaknesses under uncertainty:* it ignores g(n), so weather and terrain cost are ignored. It is not optimal. It can be trapped by dead ends (a point close in a straight line but behind a blocked bridge). It knows nothing about unreported blockages and commits to a direction.

ii. A* (f(n) = g(n) + h(n))

A* adds the cost already paid to the estimate of the cost remaining. g(n) pushes the drone away from expensive or risky legs, and h(n) pulls it toward the goal. Example: the ridge route is 8 km but a storm triples its cost (cost 24); the valley route is 12 km at cost 12. Greedy takes the ridge because it is closer in distance. A* compares f-values and chooses the valley route. Because straight-line distance never overestimates the real cost, h is admissible, so A* is complete and optimal.

iii. Impact of heuristic accuracy

- *Greedy:* depends on h completely. A good h gives a near-direct path. A misleading h (it ignores rivers, blocked roads, weather) gives long detours or failure.
- *A*:* a more accurate h (close to the true cost without exceeding it) means fewer nodes expanded. A weak h (for example h = 0) makes A* behave like Dijkstra: still optimal but slow. If h overestimates, A* can miss the optimal path. Straight-line distance ignores weather, so it is safe but weak; multiplying it by a weather factor helps only if it never overestimates.

iv. Effect of dynamic changes (blocked paths)

A blocked path makes the current plan invalid. Greedy simply picks the next best neighbour from where it is, so it reacts instantly but may loop or hit another dead end. A* must replan from the drone's current position, which costs more computation. Incremental versions (D* Lite, LPA*) reuse the earlier search and update only the affected edges, so replanning stays fast. Search time grows with the number of blocked edges and how often updates arrive.

v. Recommendation

Criterion	Greedy	A*
Uses real cost (weather/terrain)	No	Yes
Optimal	No	Yes (admissible h)
Speed per decision	Fastest	Slower, can be bounded
Safety/risk awareness	Poor	Good (risk added to g)
Handles blocked paths	Quick but short-sighted	Replans (D* Lite makes it efficient)

Decision: Use A* with an admissible, weather-aware heuristic and replanning (D* Lite style) each time new blocked-path data arrives. It gives the lowest-delay, lowest-risk route and stays safe because risk is part of g. Use Greedy only as an emergency fallback if the time budget for planning runs out.

Question 2 – Smart-city traffic signal optimisation

Formulation as an optimisation problem

- *State:* the timing plan for all intersections (cycle length, green split, offset for each).
- *Objective function:* minimise f = w1 × total waiting time + w2 × fuel consumption + w3 × congestion index.
- *Neighbours:* change one intersection's green time or offset by a small step (for example ±5 s).
- *Constraints:* minimum green time for pedestrians, maximum cycle length, emergency-vehicle priority.

Why traditional search is inefficient: With hundreds of intersections and many timing choices each, the space is astronomically large (for example 10 options × 300 intersections = 10^300 states). There is no single goal state, because we want the best plan rather than a specific one, and the path to the answer does not matter. Traffic changes constantly, so a stored plan goes stale. BFS, DFS and A* would need impossible memory for their frontier, so local search is the right family.

i. Hill Climbing

Start from the current timings, evaluate neighbouring plans, move to the best neighbour that lowers f, and stop when no neighbour is better. It is simple, fast, uses almost no memory and can run continuously online. *Limitations:* it gets stuck at local optima, wanders on plateaus, struggles on ridges, never backtracks, depends on the starting plan and step size, and copes poorly when the traffic pattern shifts the landscape.

ii. Local maxima, plateaus, ridges (real-world meaning)

- *Local maximum:* every intersection is tuned well on its own, and changing any single light makes waiting time worse, yet a coordinated corridor change (a "green wave") would be much better.
- *Plateau:* at night or off-peak, small timing changes do not change waiting time at all, so the algorithm has no direction to follow.
- *Ridge:* the improvement needs two or more lights to change together (the offset and green split of consecutive intersections). Single-light moves look worse in every direction, so progress zig-zags or stalls.

iii. How Simulated Annealing and Genetic Algorithms overcome these

- *Simulated Annealing:* pick a random neighbour. Accept it if it is better. If it is worse, accept it with probability e^(−Δ/T). At high temperature T the system accepts temporary increases in waiting time, which lets it escape local maxima and cross plateaus. As T cools it settles into a good solution. It can be reheated when traffic patterns change.
- *Genetic Algorithm:* keep a population of complete timing plans. Fitness = −f, evaluated with a traffic simulation. Selection keeps good plans. Crossover swaps whole corridors or zones between good plans, which handles ridges because many lights change together. Mutation adds random tweaks to keep diversity. Many candidates are evaluated in parallel, which suits large problems.

iv. Recommendation

Decision: Use a Genetic Algorithm as the main method, with hill climbing as a quick local refinement (a hybrid). *Scalability:* population search is parallelisable and handles hundreds of intersections. *Adaptability:* when traffic changes, start the next generation from the previous best plans instead of restarting. *Quality:* crossover over corridors handles ridges, and diversity avoids getting trapped in one local optimum. Simulated Annealing is a good lighter alternative where compute is limited, because it needs only one solution at a time.

Question 3 – Autonomous Mars rover (online search)

i. Why an online search agent is needed

Offline search needs the full map (states, actions, costs) before moving, and the rover does not have one. An online agent interleaves planning and acting: sense, decide a move, move, sense again,

update. This is also necessary because radio delay to Earth is minutes long, so the rover cannot be remote-controlled, and because a wrong move (a crater, a stuck wheel) cannot be undone.

ii. Challenges of partial observability and dynamic environments

- *Partial observability:* sensors only see a limited radius; dust and shadows hide hazards; similar-looking terrain can be safe or dangerous. The rover acts on incomplete, noisy percepts and cannot be sure where it is (localisation drift).
- *Dynamic environment:* dust storms change visibility, loose soil shifts, wheel slip changes travel cost, and energy and time are limited.
- *Consequences:* plans become invalid, some actions are irreversible, dead ends can trap the rover, and efficiency is judged against what an agent with a full map would have done.

iii. Building and updating the internal model

The rover keeps an occupancy/cost map where each cell is *unknown*, *free*, *obstacle* or *hazard* (with a probability), plus a pose estimate from odometry and landmarks. At every step: (1) sense the cells within the sensor radius; (2) update the map, marking craters and rocks and revising slope and terrain costs; (3) update stored heuristic values, as in LRTA* where the value of the cell just left becomes the best neighbour's value plus step cost; (4) plan on the current map, giving unknown cells a cost penalty; (5) execute only the first step, then repeat.

iv. Online vs uninformed vs informed search

Feature	Uninformed (BFS/DFS/UCS)	Informed (A*/Greedy)	Online (LRTA*, online DFS, D* Lite)
Needs full map in advance	Yes	Yes	No
Uses heuristic	No	Yes	Yes (and learns it)
Plans before acting	Yes	Yes	Plans a step, acts, replans
Handles new information	Poorly (restart)	Poorly (restart)	Yes, by design
Safety on unknown terrain	None	None	Can add caution and hazard costs

Uninformed and informed search are offline: they assume a known state space, so they cannot be used as they are on an unknown planet. Online search uses the same ideas (heuristics, cost) but applies them step by step.

v. Most suitable approach

Decision: An informed online search such as D* Lite (or LRTA* for very small memory), guided by a heuristic toward the sample site, with a safety layer. *Uncertainty:* it updates the model with every percept. *Adaptability:* D* Lite repairs the plan incrementally when a new obstacle appears, instead of replanning from scratch. *Safety:* unknown cells get a high cost, the rover only enters cells confirmed safe, avoids irreversible moves, and always keeps enough energy to return.

Question 4 – University exam timetable (CSP)

Formulation as a CSP

- *Variables:* for every exam e, a time slot T_e and a hall R_e.
- *Domains:* T_e ∈ {1 … S} (for example 10 days × 3 sessions = 30 slots), minus slots when the faculty is unavailable. R_e ∈ the set of halls with enough capacity (or accessible halls for exams that need them).
- *Constraints (with explanation):*
- 1. *No student clash:* if two exams share a student, T_e1 ≠ T_e2 (a binary constraint; this is graph colouring).
- 2. *Hall limit:* exams in the same slot cannot share a hall, and total enrolment in a hall must not exceed its capacity.

- 3. *Precedence:* if subject A must come before B, T_A < T_B.
- 4. *Faculty:* one examiner/invigilator cannot be in two exams at once (T_e1 ≠ T_e2 for the same faculty); unavailable slots are removed from the domain (unary constraint).
- 5. *Special accommodations:* students needing extra time or accessible rooms restrict R_e and need a longer slot or no back-to-back exams.
- 6. *Soft constraints:* avoid three exams in one day for a student and spread exams evenly (handled as preferences).

Worked mini-example: Courses Maths (M), Physics (P), AI (A), DB (D); slots {1,2,3}. M, P, A share students (M ≠ P, M ≠ A, P ≠ A). A must be before D (A < D).

ii. Backtracking search

Backtracking assigns one variable at a time (depth-first), checks the constraints against what is already assigned, and undoes the last assignment on failure. Example: M=1 → P=1 fails, P=2 → A=1 and A=2 fail, A=3 → D must be > 3, no value, so backtrack. A has no other value, so undo P and try P=3 → A=2 → D=3. Solution: M=1, A=2, P=3, D=3. It works, but it wasted effort on the dead end.

iii. Constraint propagation (forward checking)

Forward checking removes values from the domains of unassigned neighbours as soon as a variable is assigned, and backtracks immediately if a domain becomes empty. In the example, arc consistency on A < D first prunes A to {1,2} and D to {2,3}. After M=1, forward checking leaves P {2,3} and A {2}. A=2 forces P=3, and D>2 gives D=3. No backtracking is needed. Add MRV (choose the exam with the fewest legal slots first), the degree heuristic (the exam sharing students with the most others) and least-constraining-value to order the search.

iv. Real-world challenges and best strategy

Timetabling is NP-hard: thousands of courses and students make plain backtracking too slow. Constraints can conflict (over-constrained instances), soft preferences compete, and last-minute changes (an examiner drops out) need quick repair.

Decision: Use CSP backtracking with MRV/degree ordering and constraint propagation (forward checking or MAC) to find a *feasible* timetable that satisfies all hard constraints. Then improve soft constraints and repair later changes with local search (the min-conflicts heuristic), and decompose by department or exam group to scale. This keeps correctness for the hard rules while staying scalable and flexible.

Question 5 – Real-time strategy game AI (Minimax and Alpha-Beta)

i. How Minimax models the problem

Treat the game as a two-player zero-sum game with discrete decision steps. MAX is the AI, MIN is the human, assumed to play the best move against the AI. Each node is a game state and each edge a move (build, attack, move a squad). Leaves are terminal states or depth-limited states scored by an evaluation function. MAX nodes take the largest child value, MIN nodes the smallest, and the AI plays the root move with the best backed-up value.

Example tree: the root (MAX) has three moves. Their MIN nodes have leaves [3, 12, 8], [2, 4, 6], [14, 5, 2]. The MIN values are 3, 2, 2, so MAX picks the first move (value 3).

ii. Computational challenges

The number of nodes is about b^d (branching factor b, depth d). With b = 30 and d = 6 that is about 7 × 10^8 states, far too many for a decision limit of under a second. Other issues: expensive evaluation of each state, memory, imperfect information (fog of war), simultaneous moves, and the horizon effect (missing a threat just beyond the search depth).

iii. How Alpha-Beta pruning improves efficiency

Alpha is the best value MAX can already guarantee; beta is the best value MIN can already guarantee. When alpha ≥ beta, the rest of that branch cannot change the final decision, so it is skipped. The result is exactly the same as minimax (no loss of optimality).

On the example tree: the first MIN node gives 3, so alpha = 3 at the root. In the second MIN node the first leaf is 2, so beta = 2 ≤ alpha = 3, and the leaves 4 and 6 are pruned. The third node is fully checked (14, 5, 2 → 2). The answer is still 3, with 2 of 9 leaves skipped. With perfect move ordering the work drops from O(b^d) to about O(b^(d/2)), which roughly doubles the depth searched in the same time. Good move ordering matters: try captures and attacks first, and use killer moves and the best move from the previous iteration.

iv. Evaluation function design

Eval(s) = w1·(army strength, me − opponent) + w2·(resource income and stockpile) + w3·(map and territory control) + w4·(base safety: health of key buildings) + w5·(tech and upgrades) + w6·(unit positioning and mobility).

- *Army strength* is the strongest predictor of winning fights.
- *Economy* decides the future army and the long-run outcome.
- *Map control* gives vision, expansion options and resource access.
- *Base safety* captures the real win/lose condition.
- *Tech and positioning* reflect long-term and tactical advantage.

The features must be cheap to compute (they are called millions of times) and correlated with winning. The weights are tuned from game data or self-play.

v. Strategy balancing optimality and real-time performance

Decision: Use iterative deepening alpha-beta with a time limit. Search depth 1, then 2, then 3 and so on, and when time is up play the best move from the last *completed* depth, so there is always a valid answer. Improve speed with move ordering, a transposition table (a cache of repeated states) and forward pruning (only the top-k promising moves, or squads instead of single units to cut the branching factor). Use quiescence search to avoid the horizon effect, and fall back to a simple scripted move if time runs out. This gives near-optimal play within the strict time limit. For very large branching factors or hidden information, Monte Carlo Tree Search is a good alternative.